

Mobile Android — Kotlin

App Android nativa con **Kotlin 2.2** y **Jetpack Compose**, arquitectura limpia con separación Remote/Repository, **Hilt** para inyección de dependencias, **Room** para persistencia local y **Supabase KT** para autenticación y tiempo real.

Comandos principales

```
./gradlew installDebug      # Instala en dispositivo o emulador
./gradlew assembleDebug    # Genera APK debug
./gradlew assembleRelease  # Genera APK/AAB release
./gradlew test              # Tests unitarios (JUnit)
./gradlew connectedAndroidTest # Tests instrumentados (Espresso)
./gradlew tasks             # Muestra todas las tareas
```

También se puede abrir en **Android Studio** y ejecutar la configuración `app` directamente.

Stack

Capa	Tecnología	Detalle
Lenguaje	Kotlin 2.2	Coroutines, Flow, KSP
UI	Jetpack Compose	Declarativa, Material 3
Navegación	Navigation Compose	Type-safe routes
DI	Hilt + KSP	Inyección de dependencias
Persistencia local	Room	Caché offline, generaciones IA
Auth / Realtime	Supabase KT	Auth, Storage, Realtime
API GraphQL	Retrofit + OkHttp	Cliente HTTP, tipado con data classes
Cámara	CameraX	Captura de fotos para posts
Tareas diferidas	WorkManager	Recordatorios, subida diferida
SDK mínimo	minSdk 34	Android 14+
SDK objetivo	targetSdk / compileSdk 36	Android 16

Arquitectura

```
flowchart TD
    subgraph UI ["Capa UI (ui/)"]
        Screens["Screens\n(Jetpack Compose)"]
        ViewModels["ViewModels\n(StateFlow / UiState)"]
        Nav["Navigation\n(Routes.kt + NavGraph)"]
    end

    subgraph Domain ["Capa de Dominio"]
        Repos["Repositories\n(lógica de negocio)"]
        Models["Domain Models\n(Post, User, Recipe...)"]
    end

    subgraph Data ["Capa de Datos (data/)"]
        Remote["Remote Data Sources\n(Supabase KT + Retrofit/GraphQL)"]
        Local["Local Data Sources\n(Room DAOs)"]
    end

    subgraph DI ["Inyección de Dependencias (di/)"]
        Hilt["Hilt Modules\n(Network · Database · Supabase · App)"]
    end

    subgraph External ["Externo"]
        SB["Supabase\nAuth · Storage · Realtime"]
        GQL["API GraphQL\n(Cloudflare Worker)"]
    end

    Screens --> ViewModels
    ViewModels --> Repos
    Repos --> Remote
    Repos --> Local
    Remote --> SB
    Remote --> GQL
    Hilt -.→|"provee"| Remote
    Hilt -.→|"provee"| Local
    Hilt -.→|"provee"| Repos
```

Patrón Remote vs Repository

Remote Data Source — "Habla con la API"

Solo hace peticiones. Sin lógica de negocio. Devuelve exactamente lo que recibe.

```
class PostRemoteDataSource @Inject constructor(
    private val supabase: SupabaseClient,
    private val graphqlService: GraphQLService
) {
    // Solo pregunta a Supabase/GraphQL
    suspend fun getPostById(id: String): PostDto =
        supabase.from("posts").select().eq("id", id).decodeSingle()

    suspend fun getFeed(first: Int, after: String?): FeedConnection =
        graphqlService.query(FeedQuery(first, after))
}
```

Repository — "Tiene lógica de negocio"

Combina remote + local, aplica reglas, maneja errores.

```
class PostRepository @Inject constructor(
    private val remote: PostRemoteDataSource,
    private val localDao: PostDao
) {
    // Lógica de negocio: caché offline, errores controlados, transformación
    suspend fun getPost(id: String): Result<Post> = runCatching {
        localDao.getById(id)
        ?: remote.getPostById(id).toDomain().also { localDao.insert(it.toEntity()) }
    }

    fun getFeedPager(): Flow<PagingData<Post>> = Pager(
        config = PagingConfig(pageSize = 10)
    ) { FeedPagingSource(remote) }.flow
}
```

Regla: los ViewModels solo conocen Repositories. Los Repositories son los únicos que conocen Remote y Local.

Estructura del proyecto

```
app/src/main/java/es/PapayaSA/letmecook/
├─ data/
│   ├─ model/
│   │   └─ Post.kt, User.kt, Recipe.kt, Message.kt...    ← Domain models
│   │   └─ dto/    PostDto.kt, UserDto.kt...            ← DTOs de red
│   └─ remote/
│       └─ PostRemoteDataSource.kt
│       └─ UserRemoteDataSource.kt
│       └─ GraphQLService.kt                            ← Interface Retrofit
│   └─ repository/
│       └─ PostRepository.kt
│       └─ UserRepository.kt
│       └─ RecipeRepository.kt
│   └─ local/
│       └─ AppDatabase.kt                                ← Room DB
│       └─ dao/    PostDao.kt, RecipeDao.kt...
├─ di/
│   └─ SupabaseModule.kt    ← SupabaseClient singleton
│   └─ NetworkModule.kt     ← Retrofit + OkHttp singleton
│   └─ DatabaseModule.kt    ← Room DB + DAOs
│   └─ AppModule.kt        ← Repositorios y otros
├─ ui/
│   └─ navigation/
│       └─ Routes.kt        ← Definición de rutas
│       └─ NavGraph.kt      ← Grafo de navegación Compose
│   └─ screens/             ← Una carpeta por pantalla
│       └─ home/    HomeScreen.kt + HomeViewModel.kt
│       └─ auth/    LoginScreen.kt + RegisterScreen.kt
│       └─ profile/  ProfileScreen.kt + ProfileViewModel.kt
│       └─ ...
│   └─ theme/
│       └─ Color.kt, Typography.kt, Shape.kt
│       └─ Theme.kt        ← MaterialTheme
│   └─ components/         ← Composables reutilizables
```

```
├─ utils/
|   ├─ Constants.kt      ← URL base, claves
|   ├─ ImageUtils.kt
|   └─ LanguageUtils.kt
└─ worker/
    ├─ ReminderWorker.kt  ← Recordatorios de recetas
    └─ PostUploadWorker.kt ← Subida diferida de posts
```

Pantallas y navegación

```
flowchart TD
    Login --> Register
    Login --> CompleteProfile
    Login --> Home

    Home --> PostDetail
    Home --> UserProfile
    Home --> Camera

    subgraph Main ["Navegación principal (Bottom Bar)"]
        Home["Home\nFeed principal"]
        Explore["Explore\nDescubrir posts"]
        Generator["Generator\nRecetas IA"]
        Messages["Messages\nConversaciones"]
        Profile["Profile\nPerfil propio"]
    end

    Explore --> PostDetail
    Explore --> UserProfile
    Profile --> Settings
    Profile --> Following
    Profile --> Followers
    Messages --> Conversation
```

Ruta Compose	Pantalla	Descripción
Login	Inicio de sesión	Email / contraseña / Google
Register	Registro	Alta de cuenta nueva
CompleteProfile	Completar perfil	Tras alta o login social
Home	Feed principal	Posts de usuarios seguidos
Explore	Explorar	Descubrir posts y usuarios
Generator	Generador IA	Generar recetas con IA
Profile	Perfil propio	Posts, bio, seguidores
UserProfile(userId)	Perfil ajeno	Perfil público
Camera(mode)	Cámara	CameraX para fotos de posts
PostDetail(postId)	Detalle post	Receta completa, comentarios
Notifications	Notificaciones	Likes, follows, menciones
Messages(pendingUserId)	Mensajes	Lista de conversaciones
Conversation(conversationId, otherUserId, otherUserName)	Chat	Conversación directa
Following(userId)	Siguiendo	Lista de seguidos
Followers(userId)	Seguidores	Lista de seguidores

Módulos Hilt

```
// SupabaseModule.kt
@Module @InstallIn(SingletonComponent::class)
object SupabaseModule {
    @Provides @Singleton
    fun provideSupabase(): SupabaseClient = createSupabaseClient(
        supabaseUrl = BuildConfig.SUPABASE_URL,
        supabaseKey = BuildConfig.SUPABASE_ANON_KEY
    ) {
        install(Auth)
        install(Realtime)
        install(Storage)
    }
}

// NetworkModule.kt
@Module @InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides @Singleton
    fun provideRetrofit(): Retrofit = Retrofit.Builder()
        .baseUrl(Constants.URL) // "https://develop.app.savorealo.com/api/"
        .addConverterFactory(GsonConverterFactory.create())
        .client(OkHttpClient.Builder()
            .addInterceptor(AuthInterceptor()) // añade JWT
            .build())
        .build()
}
```

Configuración de URLs

`utils/Constants.kt` :

```
// Producción
const val URL = "https://develop.app.savorealo.com/api/"

// Backend local en emulador Android
const val URL = "http://10.0.2.2:8787/"

// Backend local en dispositivo físico (IP LAN del PC)
const val URL = "http://192.168.1.XX:8787/"
```

Retrofit construye el endpoint final: `${URL}graphql`

Workers (tareas en background)

Worker	Trigger	Descripción
<code>ReminderWorker</code>	<code>WorkManager</code> periódico	Notifica cuando hay recetas guardadas
<code>PostUploadWorker</code>	<code>WorkManager</code> + constraints de red	Sube posts cuando hay conectividad

```
// Enqueue con constraint de red
val uploadRequest = OneTimeWorkRequestBuilder<PostUploadWorker>()
    .setConstraints(
        Constraints.Builder().setRequiredNetworkType(NetworkType.CONNECTED).build()
    )
    .setInputData(workDataOf("postId" to postId))
    .build()
WorkManager.getInstance(context).enqueue(uploadRequest)
```

Build y despliegue

```
./gradlew assembleDebug    # APK debug (development)
./gradlew assembleRelease  # APK/AAB release (requiere keystore)
```

Configuración en `app/build.gradle.kts`:

Parámetro	Valor
<code>applicationId</code>	<code>es.PapayaSA.letmecook</code>
<code>compileSdk</code>	36
<code>minSdk</code>	34 (Android 14)
<code>targetSdk</code>	36

Antes de publicar: configurar firma con keystore, mover claves de Supabase/Google a `secret.properties` o BuildConfig secrets, revisar reglas ProGuard/R8.